

Computer Graphics

Mathematical Fundamentals

Matrix Math

1	2	3	4
5	6	7	8

2x4

- Matrix dimensions are specified as:
rows x cols

Matrix Addition

$$\begin{array}{|c|c|} \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} + \begin{array}{|c|c|} \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} = \begin{array}{|c|c|} \hline 2 & 4 \\ \hline 6 & 8 \\ \hline \end{array}$$

2x2 2x2 2x2

- The dimensions must match to add

Scalar Multiplication

$$5 \times \begin{array}{|c|c|} \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} = \begin{array}{|c|c|} \hline 5 & 10 \\ \hline 15 & 20 \\ \hline \end{array}$$

- The dimensions can be any size to perform scalar multiplication

Matrix Multiplication

$$\begin{array}{c}
 \text{row 1} \\
 \begin{array}{|c|c|c|}
 \hline
 1 & 2 & 3 \\
 \hline
 4 & 5 & 6 \\
 \hline
 \end{array}
 \end{array}
 \times
 \begin{array}{c}
 \text{col 2} \\
 \begin{array}{|c|c|c|}
 \hline
 1 & 2 & 3 \\
 \hline
 4 & 5 & 6 \\
 \hline
 7 & 8 & 9 \\
 \hline
 \end{array}
 \end{array}
 =
 \begin{array}{c}
 \text{row 1, col 2} \\
 \begin{array}{|c|c|c|}
 \hline
 30 & 36 & 42 \\
 \hline
 66 & 81 & 96 \\
 \hline
 \end{array}
 \end{array}$$

$2 \times 3 \quad 3 \times 3 \quad 2 \times 3$

$$1 \times 2 + 2 \times 5 + 3 \times 8 = 2 + 10 + 24 = 36$$

- rows from the first matrix
- columns from the second matrix

Matrix Multiplication

- “inner” dimensions must match
- result is “outer” dimension
- Examples:
 - $2 \times 3 \times 3 \times 3 = 2 \times 3$
 - $3 \times 4 \times 4 \times 5 = 3 \times 5$
 - $2 \times 3 \times 4 \times 3 = \text{cannot multiply}$
- Question: Does $AB = BA$?
 - Hint: let A be 2×3 and B be 3×2

Matrices and Graphics

- Matrices are used to represent equations
- Used as an efficient way to represent transformation equations in graphics
 - Multiple transformations can be composed into a single matrix
 - Matrix multiplications are often optimized in hardware

Transformations

- View a complex shape as a set of points
- To transform the shape, one transforms each of the points that define the shape
- Major types of transformations:
 - Translation
 - Rotation
 - Scaling

Translation

- The equation to translate a single point (x, y) by t_x in the x direction and t_y in the y direction:

$$x' = x + t_x$$

$$y' = y + t_y$$

Resulting in the new point (x', y')

Translation

- These equation can be represented using matrices:

$$P = \begin{bmatrix} x \\ y \end{bmatrix} \quad P' = \begin{bmatrix} x' \\ y' \end{bmatrix} \quad T = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

- Then using matrix math we simply have:

$$P' = P + T$$

Rotation

- The equations to rotate a point through an angle, θ (ccw), around the origin:

$$x' = x \cos(\theta) - y \sin(\theta)$$

$$y' = x \sin(\theta) + y \cos(\theta)$$

- In matrix form:

$$R = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

- Using matrix math: $P' = R P$

Scaling

- The equations to scale a point along the coordinate axes:

$$x' = x s_x$$

$$y' = y s_y$$

- Using matrix math: $P' = S P$

- Where $S =$  $?$

Scaling

- The equations to scale a point along the coordinate axes:

$$x' = x \cdot s_x$$

$$y' = y \cdot s_y$$

- Using matrix math: $P' = S P$
- Where $S =$  $?$

Scaling

- Scaling is used to grow/shrink objects centered on the origin
- s numbers bigger than 1 grow objects
- s numbers smaller than 1 shrink objects
- uniform scaling: both s numbers the same

Transformation Sequences

- In general you need to perform a sequence of transformation:

This is a rotation followed by a translation

$$P' = R P + T$$

Do a dimension check to make sure the math will work out:

- R is 2x2 and P is 2x1, thus R P is 2x1
- R P is 2x1 and T is 2x1, thus P' is 2x1

Transformation Sequences

- However, it is often more efficient if we can combine operations into a single matrix calculation
- How can this be done?
 - Here are the equations:

$$x' = (x \cos(\theta) - y \sin(\theta)) + t_x$$

$$y' = (x \sin(\theta) + y \cos(\theta)) + t_y$$

Transformation Sequences

- Put them into matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & t_x \\ \sin(\theta) & \cos(\theta) & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- The only problem is that our original point, P, now need a 3x1 matrix while the resulting point, P', needs a 2x1 matrix

Homogeneous Coordinates

- We resolve this using homogeneous coordinates
- Lots of theory here, but...
- Basically boils down to all points being represented as a triple (x, y, 1)
- Thus, we need to fix our transformation matrix so that P' comes out as a triple

Homogeneous Coordinates

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & t_x \\ \sin(\theta) & \cos(\theta) & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- This 3x3 matrix is called a transformation matrix
 - This particular matrix rotates then translates
- However, it is often easier to work with transformations matrices that handle one type of transformation at a time, then compose them to obtain our final transformation

2D Transformation Matrices

- Translation: $T(x, y)$

1	0	t_x
0	1	t_y
0	0	1

- Rotation: $R(\theta)$

$\cos(\theta)$	$-\sin(\theta)$	0
$\sin(\theta)$	$\cos(\theta)$	0
0	0	1

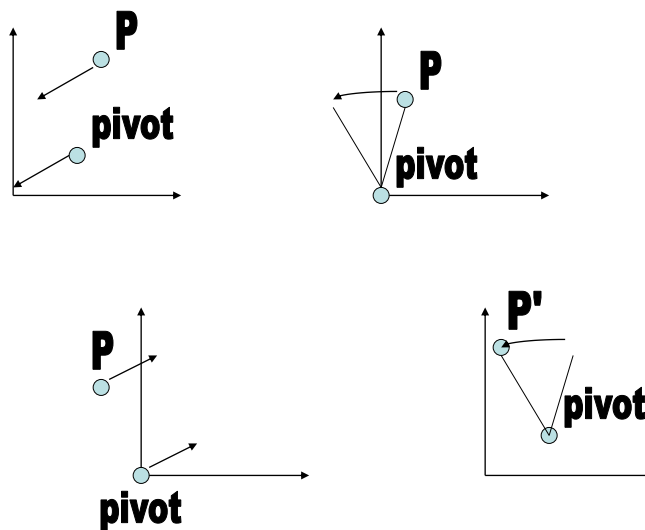
- Scaling: $S(x, y)$

s_x	0	0
0	s_y	0
0	0	1

Transformation Composition

- Example: How do you rotate point P about a random “pivot point”?
 - Note that normal rotation has the origin as a pivot
1. Translate P so pivot point lines up with the origin
 2. Rotate P around origin
 3. Translate P back so pivot point is back on original position

Rotation Around Pivot Point



Rotation Around Pivot Point

- Assume pivot point is at (x, y)
- Assume P is the point you wish to rotate

$$P' = T(x, y) R(\theta) T(-x, -y) P$$

– Recall that order make a huge difference!

Other Transformations

- Reflection
 - about x axis (left)
 - about y axis (right)

1	0	0
0	-1	0
0	0	1

-1	0	0
0	1	0
0	0	1

- Shear
 - along x axis (left)
 - along y axis (right)

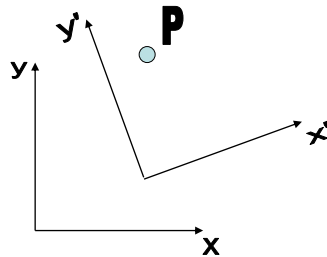
1	sh_x	0
0	1	0
0	0	1

1	0	0
sh_y	1	0
0	0	1

- Identity

1	0	0
0	1	0
0	0	1

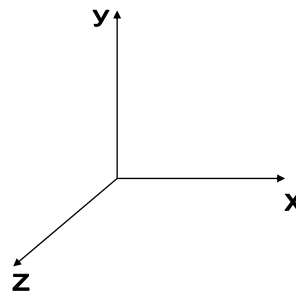
Transformation Between Coordinate Systems



- Point P is represented in the original coordinate system. To transform it to the new coordinate system one needs to know how this system is oriented and positioned relative to the original coordinate system.

3D Transformations

- I'll be using a right-handed coordinate system
 - Positive z-axis is sticking out of the screen
 - Dashed lines will be used for lines that stick into the screen



3D Transformations

- Points now have 3 coordinates (x, y, z)
- Written in homogeneous coordinates as (x, y, z, 1)
- Since the points are now 4x1 matrices, the transformation matrices need to be 4x4

3D Translation

$T(x, y, z):$

1	0	0	t_x
0	1	0	t_y
0	0	1	t_z
0	0	0	1

3D Rotation

- 3D rotation is a bit harder than in 2D
 - In 2D we always rotated about the z-axis
 - Now we have a choice of 3 axes, R_x , R_y , R_z
 - Rotation is still ccw about the axis
 - Looking from the positive half of the axis back towards the origin (same as it was in 2D)

$R_z(\theta)$

- Extending our 2D rotation equations to 3D:

$$\left. \begin{aligned} x' &= x \cos(\theta) - y \sin(\theta) \\ y' &= x \sin(\theta) + y \cos(\theta) \end{aligned} \right\} \text{ same as in 2D}$$

$$z' = z \quad \left. \vphantom{\begin{aligned} x' &= x \cos(\theta) - y \sin(\theta) \\ y' &= x \sin(\theta) + y \cos(\theta) \end{aligned}} \right\} \text{ the height stays the same}$$

$R_z(\theta)$:

$\cos(\theta)$	$-\sin(\theta)$	0	0
$\sin(\theta)$	$\cos(\theta)$	0	0
0	0	1	0
0	0	0	1

$R_x(\theta)$ and $R_y(\theta)$

$R_x(\theta)$:

1	0	0	0
0	$\cos(\theta)$	$-\sin(\theta)$	0
0	$\sin(\theta)$	$\cos(\theta)$	0
0	0	0	1

$R_y(\theta)$:

$\cos(\theta)$	0	$\sin(\theta)$	0
0	1	0	0
$-\sin(\theta)$	0	$\cos(\theta)$	0
0	0	0	1

Rotation Around a General Axis

1. Translate P so that the rotation axis passes through the coordinate system origin
2. Rotate so that the axis of rotation coincides with one of the coordinate axes
 - In general, requires 2 rotations (about the other 2 axes)
3. Rotate around the axis
4. Perform the reverse of the rotations (2)
5. Perform the reverse of the translation (1)

Rotation Around a General Axis

$$P' = T(-a, -b, -c) R_x(-\psi) R_y(-\rho) R_z(\theta) R_y(\rho) R_x(\psi) T(a, b, c) P$$

- Requires 7 matrix multiplications
 - Each one requiring 64 multiplies and 48 adds
- For a total of 448 multiplies and 336 adds
 - Per point!
 - Recall objects sometimes have >100k points

Pre-multiplication of Transformations

- Recall that matrix multiplication is not commutative: $AB \neq BA$
- However, it is associative: $(AB)C = A(BC)$
 - This implies that you don't have to perform the matrix multiplies right-to-left
- Thus, we can pre-multiply the 7 matrices that don't change between points to obtain:

$$A = T(-a, -b, -c) R_x(-\psi) R_y(-\rho) R_z(\theta) R_y(\rho) R_x(\psi) T(a, b, c)$$
- Then we repeat the single multiplication $P' = A P$ for each of the 100k points

Pre-multiplication of Transformations

- The old method produced (for 100k points)
44.8M multiplies and 33.6M adds
- The pre-multiplication method produces
approx 6.4M multiplies and 4.8M adds
- Plus, not all the multiplies/adds need to be done
since we are assuming 0 0 0 1 in the last row
- Plus, these multiplies/adds are often done on
optimized graphics hardware